

1. Goal of the System

Build a comprehensive system featuring:

- **Main Orchestration Workflow:** Decides which hotel negotiation sessions to launch.
- **Worker Workflows:** Manages individual, live hotel negotiations.
- **Shared Backend:** Ensures all workers see synchronized hotel pricing and historical behavior.
- **Long-Term Memory Layer:** Provides workers with historical beliefs, such as identifying times a hotel accepts lower rates.

Key Design Principles:

- Workflows manage the process steps and session state.
- The database serves as the shared source of truth.
- The memory layer handles learned patterns and behaviors.
- The voice layer, powered by **GPT Realtime 1.5**, handles low-latency audio.
- The reasoning layer, powered by **GPT-4o**, handles bargaining logic.

2. Core Architecture

A. Main Orchestration Workflow

This acts as the session launcher and supervisor. It is responsible for ingesting hotel targets, prioritizing calls, preventing duplicate sessions, launching worker sessions, monitoring outcomes, requeuing failed jobs, and updating the overall campaign state.

B. Worker Workflow

This manages a single live hotel call session. It is responsible for loading hotel context and long-term memory, listening to speech via GPT Realtime 1.5, reasoning about pricing facts and negotiation moves via GPT-4o, speaking back, and updating the shared backend.

C. Shared Backend

This operates outside the workflow management tool and is strictly required. It handles hotel metadata, the latest quotes, rate histories, active session locks, event logs, call outcomes, and confidence-weighted long-term memory features.

D. Real-Time Voice Pipeline

Connected to the worker workflows, this handles phone audio input, speech processing, text-to-speech output, and turn detection using GPT Realtime 1.5.

3. Main Orchestration Implementation Plan

Treat the main orchestration workflow as the central control plane.

Main State Variables

Variable	Description
Campaign ID	Identifier for the current campaign.
Target Jobs	List of initial negotiation tasks.
Queued Jobs	List of tasks ready for execution.
Active Workers	Status tracking for currently running sessions.
Completed Jobs	List of successfully finished tasks.
Failed Jobs	List of tasks that encountered errors.
Retry Jobs	Tasks scheduled for a second attempt.
Market Signals	Broader trends or data affecting the current market.
Launch Budget	Budget tracking for the campaign.
System Status	Overall health and state of the orchestration system.

Main Workflow Steps (Nodes)

1. Ingest Targets

- **Purpose:** Load hotels, dates, room classes, priorities, and customer constraints.
- **Inputs:** Campaign request, hotel list, date windows, maximum budget, and maximum sessions.
- **Outputs:** Normalized target jobs.

2. Deduplicate and Merge Targets

- **Purpose:** Remove duplicate jobs and merge overlapping requests based on the same hotel, check-in/out dates, room type, and booking goal.
- **Outputs:** A deduplicated execution queue.

3. Load Shared Market State

- **Purpose:** Query the shared database for the latest hotel quotes, active locks, stale quote timestamps, current market signals, and known call restrictions.
- **Outputs:** Enriched queued jobs.

4. Score and Prioritize Jobs

- **Purpose:** Rank jobs by value based on expected savings, quote staleness, booking urgency, probability of successful negotiation, customer importance, and time-of-day value.
- **Outputs:** A prioritized execution queue.

5. Check Launch Eligibility

- **Purpose:** Decide whether each queued job should launch based on active workers for that segment, quote recency, concurrency limits, voice capacity, and customer budget status.
- **Outputs:** Lists of launchable jobs and deferred jobs.

6. Spawn Worker Sessions

- **Purpose:** Launch worker runs for each eligible job, create a unique thread ID (combining hotel ID, dates, room type, and attempt number), store worker metadata, and mark the session as active in the shared database.
- **Outputs:** Updated active worker state.

7. Monitor Worker Events

- **Purpose:** Consume updates from workers such as call started, quote received, transferred to manager, callback requested, deal closed, failed, retry recommended, or terminated.
- **Outputs:** Updated active, completed, and failed states.

8. Retry, Requeue, or Close

- **Purpose:** Determine next steps after worker completion based on rules: retry if no valid quote, schedule reattempt if callback requested, close if quote is acceptable, or mark exhausted if unreachable after a threshold.
- **Outputs:** The retry queue, completed jobs list, and final campaign state.

9. Emit Campaign Summary

- **Purpose:** Produce a final summary including best quotes found, pending active negotiations, retry schedules, unreached hotels, and estimated savings.

Main Workflow Sequence

The sequence proceeds from ingesting targets, deduplicating, loading market state, scoring, checking eligibility, spawning workers, monitoring events, and handling retries/closures. The workflow then either loops back to scoring and prioritizing or concludes by emitting the campaign summary.

4. Worker Implementation Plan (Dual-Model Setup)

Treat the worker workflow as the live execution loop.

Model Delegation:

- **GPT Realtime 1.5:** Exclusively handles all sensory input/output (listening, detecting turns, transcribing, and speaking).
- **GPT-4o:** Exclusively handles all reasoning, logic, state evaluation, and script generation for bargaining.

Worker State Variables

Variable	Description
Thread ID	Unique identifier for the specific session attempt.
Hotel ID	Target hotel identifier.
Check-in / Check-out Date	Target dates for the negotiation.

Room Type	The specific room category being negotiated.
Session Status	Current state of the live call.
Transcript Window	Buffer of recent conversation turns.
Latest Quote	The most recent pricing information obtained.
Fees & Restrictions	Any extra costs or booking limitations discovered.
Objections Used	List of negotiation tactics already deployed.
Negotiation Strategy	The current bargaining plan determined by GPT-4o.
Manager Reached	Boolean tracking if the call was escalated.
Callback Requested	Boolean tracking if the hotel asked to call back.
Continue Call	Boolean indicating if the loop should keep running.
Next Action	The strategic move chosen by GPT-4o.

Final Outcome	The concluding result of the session.
----------------------	---------------------------------------

Worker Workflow Steps (Nodes)

1. Load Hotel Context

- **Purpose:** Load structured info from the database, including hotel name, phone number, historical quotes, prior outcomes, and date-specific details.
- **Outputs:** Initial session context.

2. Load Long-Term Memory

- **Purpose:** Fetch distilled behavioral features like acceptance price bands, time-of-day flexibility, fee-waiver probability, escalation success rates, and weekend/weekday patterns.
- **Outputs:** Behavioral priors for the strategy step.

3. Acquire Session Lock

- **Purpose:** Prevent duplicate workers on the same hotel/date band using a database lease or fast-cache lock. Terminate early if unacquired.
- **Outputs:** Lock acquired or not acquired status.

4. Start Voice Session (GPT Realtime 1.5)

- **Purpose:** Initialize telephony, connect the session, attach GPT Realtime 1.5 for audio processing, and begin listening.
- **Outputs:** Session connected status.

5. Listen for Turn (GPT Realtime 1.5)

- **Purpose:** GPT Realtime 1.5 receives the speech chunk, detects the end-of-turn, streams the transcript, and buffers the window.
- **Outputs:** The latest utterance from the hotel.

6. Extract Structured Facts (GPT-4o)

- **Purpose:** GPT-4o analyzes the transcript to extract the quoted rate, taxes, fees, room type, cancellation terms, discount authority, refusals, and negotiation openness.
- **Outputs:** Structured factual data.

7. Write Shared Quote Update

- **Purpose:** Immediately sync new quotes, timestamps, confidence scores, and restrictions to the shared database so other sessions can see fresh information.
- **Outputs:** Backend updated status.

8. Check Cross-Session State

- **Purpose:** Read the latest shared truth to check if another worker found a better quote, the budget was met, or the target was achieved elsewhere.
- **Outputs:** Decision to continue, modify, or terminate.

9. Decide Negotiation Move (GPT-4o)

- **Purpose:** GPT-4o evaluates the transcript, priors, and shared state to choose an action: ask for a lower rate, waive a fee, escalate to a manager, mention a competitor, accept, or close politely.
- **Outputs:** The specific next strategic action.

10. Generate Response Text (GPT-4o)

- **Purpose:** GPT-4o drafts the exact spoken reply based on the chosen action, ensuring concise, natural, and hotel-safe language.
- **Outputs:** The text response to be spoken.

11. Speak Response (GPT Realtime 1.5)

- **Purpose:** Route the text from GPT-4o to GPT Realtime 1.5 to synthesize and deliver natural speech output over the phone line.
- **Outputs:** Spoken response delivered.

12. Check Continue or Terminate

- **Purpose:** Evaluate if the loop continues. Terminate upon a final quote, hard refusal, callback request, duplicate detection, target achievement, or max turns exceeded.
- **Outputs:** Continue or stop command.

13. Post-Call Summary (GPT-4o)

- **Purpose:** GPT-4o summarizes the call, storing the final quote, negotiation path, refusal reasons, and candidate learnings.
- **Outputs:** Session outcome record.

14. Emit Long-Term Memory Candidates

- **Purpose:** Write potential learned patterns (e.g., "parking fee never waived") for offline validation.

15. Release Session Lock

- **Purpose:** Free the session lock after the worker concludes.

Worker Workflow Sequence

The main loop starts with setup and lock acquisition, then enters the active call cycle: listen (Realtime 1.5), extract facts (4o), sync quotes, check cross-session state, decide move (4o),

generate text (4o), speak (Realtime 1.5), and check termination. Upon stopping, it summarizes the call, emits memory candidates, and releases locks.

5. Shared Backend Plan

State must not be stored exclusively in the workflow memory.

Relational Database Tables (Structured Storage):

- **Hotels:** Core metadata.
- **Hotel Room Segments:** Specific room categorizations.
- **Quote Events:** Granular history of all quoted prices.
- **Latest Quotes:** The absolute most recent prices fetched.
- **Negotiation Sessions:** Logs of all launched worker threads.
- **Negotiation Turns:** Turn-by-turn transcripts and actions.
- **Session Locks:** Active barriers preventing duplicate calls.
- **Call Outcomes:** Final results of completed runs.
- **Long-Term Memory Candidates:** Unverified behavioral patterns.
- **Validated Hotel Features:** Confirmed behavioral patterns for future priors.

Fast Cache Storage (In-Memory/Key-Value):

- **Active Locks:** Rapidly checkable keys combining hotel, dates, and room type.
- **Short-Lived Quote Cache:** Rapidly checkable latest quotes.
- **Event Pub/Sub:** Worker event streams tied to the campaign identifier.

6. Orchestration Loop Behavior

Main Loop: Runs continuously to ingest pending jobs, refresh shared state, score jobs, launch eligible workers, wait for events, and manage retries/closures until the campaign is complete.

Worker Loop: Runs on a per-session basis to listen, extract facts, sync quotes, choose moves, speak, and decide whether to continue or stop until the call finishes.

7. Model Orchestration Plan

Live Worker Path (Fast Loop):

- **Audio Input/Output:** Handled strictly by **GPT Realtime 1.5** for maximum speed, turn detection, and natural vocal delivery.
- **Logic & Extraction:** Handled strictly by **GPT-4o** to ensure high-quality reasoning, rapid fact extraction, and intelligent negotiation strategies turn-by-turn.

Offline Path (Slow Loop):

After call completion, **GPT-4o** operates without latency constraints to provide deep transcript summarization, long-term feature extraction, strategy evaluation, memory candidate validation, and prior updates.

8. Deployment Plan

1. Main Orchestrator Service: Runs the main workflow, handles campaign control, worker launch decisions, monitoring, and retries.

2. Worker Runtime Service: Runs worker workflows, executes live hotel sessions, coordinates the dual-model handoffs, and writes shared updates.

3. Voice Gateway Service: Manages telephony integration, real-time audio streaming, and routes audio to GPT Realtime 1.5.

4. Memory Feature Service: Computes long-term hotel features, validates memory candidates, and refreshes retrieval summaries.

5. Event Bus: Manages worker status events, quote updates, and retry triggers.

9. Engineering Phases

Phase 1: Single Worker Proof of Concept

Prove the live negotiation loop by building one manual hotel call flow with shared quote writes and basic post-call summaries (no launcher workflow yet).

Phase 2: Shared Backend & Synchronization

Prove workers can read each other's updates by building the latest quote tables, lock system, long-term feature retrieval, and cross-session shared quote visibility.

Phase 3: Main Workflow Launcher

Automate campaign control by building queue ingestion, prioritization, worker spawn logic, and the monitoring/retry flow.

Phase 4: Multi-Worker Concurrency

Run many parallel calls safely by implementing concurrent worker execution, the event bus, duplicate prevention, and failure handling.

Phase 5: Long-Term Memory Refinement

Make workers smarter over time by adding candidate memory extraction, the offline validation step, hotel behavior priors, and retrieval summaries.

Phase 6: Optimization

Improve real-time responsiveness and cost by reducing the action space, implementing prompt caching, and refining the GPT Realtime 1.5 and GPT-4o routing logic.

10. Final Recommended Structure

- **Main Workflow:** Serves as the campaign scheduler, worker launcher, monitoring loop, and retry controller.

- **Worker Workflow:** Serves as a single live call loop handling short-term memory and the immediate negotiation session using the dual-model setup.
- **Shared Backend:** Acts as the synchronized truth for quotes, cross-session coordination, and long-term memory.
- **Fast Cache:** Acts as the lock manager, event stream, and rapid cache (not intended to be a standalone database per agent).